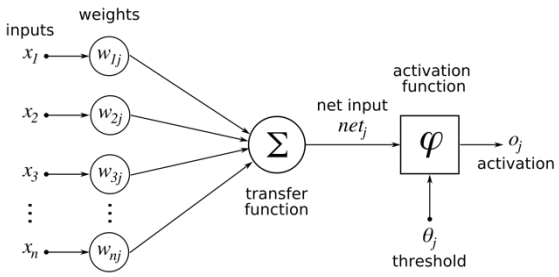
University of Southampton

ELEC Laboratory Programme

Laboratory manual

ρ3

Neural Networks



In this lab we are going to learn how to do matrix algebra in C++. We start with simple vectors then introduce a vector of vectors to serve as a matrix structure. We then need to write functions to implement matrix multiplications. Finally, we will attempt to create a rudimentary neural network using matrices.

Revision History

October 19, 2025	Dr Ivan Ling	New for 2025/26
October 26, 2025	Dr Ivan Ling	Modified in response to student feedback on lab difficulty. Added "Challenges" for students with more programming experience.
October 19, 2026	Dr Ivan Ling	Modified for more "fun factor" in optional work while making main lab easier to follow.

Electronics and Computer Science

© Electronics and Computer Science, University of Southampton

Laboratory Overview and Information

Schedule	
Preparation time:	3 hour
Lab time:	3 hours
Items provided	
Tools:	None
Components:	None
Equipment:	None
Software:	G++ Compiler
Items to bring	
Essentials:	A full list is available on the Laboratory website at https://secure.ecs.soton.ac.uk/notes/ellabs/databook/essentials/

Initial Instructions

Before you come to the lab, it is essential that you read through this document and complete ***all*** of the preparation work in section 2. Only preparation which is recorded in your laboratory logbook will contribute towards your mark for this exercise. There is no objection to several students working together on preparation, as long as all understand the results of that work. Before starting your preparation, read through all sections of these notes so that you are fully aware of what you will have to do in the lab.

Academic Integrity

If you undertake the preparation jointly with other students, it is important that you acknowledge this fact in your logbook. Similarly, you may want to use sources from the internet or books to help answer some of the questions. Again, record any sources in your logbook.

© Electronics and Computer Science, University of Southampton

1 Aims, Learning Outcomes and Outline

This laboratory exercise aims to:

- Introduce the fundamentals of matrix manipulation algorithms.
- Help you in understanding the fundamentals of encapsulation and abstraction in object-oriented programming
- Familiarise yourself with the C++ random number support

Having successfully completed the lab, you will be able to:

- Use vectors and matrices in place of arrays
- Create non-trivial data structures
- Implement a simple neural network model

For the first 2 weeks, you have had the chance to work on user-facing software projects. These projects are fun, but you will realize that in the industry, often times you will be writing codes to be used by other programmers. In these situations, your code structures, classes etc, must "make sense". In this lab, you will first create the tools and classes needed to implement matrices in C++. You will then create function overloads to support the implementation of matrix manipulations using your Matrix class. You will then create a simple neural network model to carry out a simple classification task.

The lab is heavily inspired by C3 from last year's C-programming labs. You do not need your C3 codes to get this lab working, but understanding it helps!

Lab Partners 👫
For this assignment, you may choose to collaborate with your project partner. However, it is important to understand that all students will be assessed individually.

If you work in a pair, we highly recommend a classic divide-and-conquer strategy. This is a practical opportunity to apply the team-based software development methods covered in lectures, so don't be a lone wolf!

Challenges 🚀

We understand that students have different programming abilities, and not all students have access to AI tools or choose to use AI tools. Therefore, in this lab, you could choose to implement the code based on a scaffold given or attempt to challenge yourselves by building it from scratch. **Parts of the lab where you could challenge yourselves are labelled with** 🚀.

© Electronics and Computer Science, University of Southampton

2 Preparation

Read through the course handbook statement on safety and safe working practices, and your copy of the standard operating procedure. Make sure that you understand how to work safely. Read through this document so you are aware of what you will be expected to do in the lab. You might have implemented Matrix in previous labs as a typedef of vector<vector>. In this lab, however, you will need to implement the full matrix algorithm

2.1 The groundwork

Matrix manipulation requires more work than you think. Create a class called Matrix which contains:

- A vector of vector storing doubles, ***data***
- An integer storing the number of rows, ***row***
- An integer storing the number of columns, ***col***
- A suitable constructor to initialize all the member variables

Write the class declaration in your logbook  , as this will help you in the lab work.
At the same time, read up on how to use random numbers in C++. It'll be helpful for you to write a few code examples in your logbook to assist you during the lab session.

2.2 Creating your functions

Read up on matrix addition and matrix multiplication to understand the algorithms. Make notes on your logbook detailing how the algorithm works and how you will be implementing it in your program.

Write a draft of the matrix addition and matrix multiplication code in your logbook  .

2.3 Understanding the perceptron model

Do a bit of research on multilayer perceptron and record your findings in your logbook. Your logbook record must include references where appropriate. 

- How does the multilayer perceptron work?
- What are the limitations of multilayer perceptron?
- What are the algorithms used in training a multilayer perceptron model?

Read and understand this website* and try to draw a version of the perceptron model based on your understanding. By the end of the prep, you should understand how the neural network model could be represented as simple matrix multiplications.

*https://python-course.eu/machine-learning/neural-networks-structure-weights-and-matrices.php

© Electronics and Computer Science, University of Southampton

3 Laboratory Work

3.1 Creating your backbone

Download Matrix.h and Matrix.cpp from the lab website. Populate the files with the required class and function definitions. 🚀 **For more challenge: Don't download the file and try to create the entire class yourself!**

You should also create operator overloads for the matrix operations if possible.

Write a simple test program called main.cpp to verify that your Matrix class is working as intended.

3.2 Creating your own neural net

Download the NeuralNetwork.h and NeuralNetwork.cpp file from the lab website. Use your Matrix library to populate the file with the required class and function definitions. 🚀 **For more challenge: Don't download the file and try to create the entire class yourself!**

Write a simple test program to make sure that each of the functions are working well. Pay special attention to the functions requiring matrix manipulations. You do not need to verify that the neural net can be trained. You are simply required to check if there are any errors in the code.

3.3 Completing the Neural Network

Complete the function called update() within your NeuralNetwork class

1. Take in an input matrix as the **target**.
2. Calculate the error between the target and the output of your network.
    a. Hint: The output is simply the activation from the last layer in your neural network!
3. Calculate the loss using your error matrix. You should find some way to condense the error matrix into a single number that represents your model's **loss.**
4. Update the weights using the update() function based on the errors.
5. For each layers in your model you will need to do the following:
    a. Get the current activation (output) of the layer. **Hint:** If your loop index is i, the corresponding output is at activations[i + 1]
    b. Calculate the **derivative** of the output (use the dsigmoid function provided)
    c. Multiply this derivative by the error (from the *next* layer) using **element-wise multiplication**.
    d. Finally, scale this **gradient** by the training_rate. (The gradient in this case is essentially how much we're "blaming" this neuron for the given error.)
6. Next, you will need to calculate the "delta" for the weights. This tells you how much to change each weight:
    a. Get the activated output from the previous layer (at activations[i]).
    b. Transpose this previous layer's output.

© Electronics and Computer Science, University of Southampton

   c. Calculate the delta weights by performing a **matrix multiplication** of the gradient and the transposed previous layer output.

7. Now that you have the "delta" you know exactly how much to change your weights. You should now update the current layer's weight by adding the delta directly. You should also remember to update the bias using the scaled **gradient** from step 5(d) (The bias delta is just the gradient itself!).
8. Now that we're done with the layer, pass the error to the next loop iteration to use (i.e. the previous layer, as backpropagation calculates the error from the layer closest to the output first):
   a. Transpose the current layer's weights
   b. The **error matrix** to be passed to the previous layer is just the scaled **gradient** from the current layer multiplied with the **transposed weights**

Now that your update() function is working, you should be able to write a "training" loop in the main function. The update() function should ideally take in a target and spit out a "loss" value. Your training loop should monitor this loss. An example training program is described in the next section.

```
EPOCH 16729, cross_entropy = 0.000146543
EPOCH 16730, cross_entropy = 0.000146481
EPOCH 16731, cross_entropy = 0.000146419
EPOCH 16732, cross_entropy = 0.000146358
EPOCH 16733, cross_entropy = 0.000146296
EPOCH 16734, cross_entropy = 0.000146234
EPOCH 16735, cross_entropy = 0.000146173
EPOCH 16736, cross_entropy = 0.000146111
EPOCH 16737, cross_entropy = 0.000146049
EPOCH 16738, cross_entropy = 0.000145988
EPOCH 16739, cross_entropy = 0.000145926
```

Error should go down slowly at each iteration

3.4 Train the neural net

Using the perceptron created, train the perceptron model to perform the function of a 4-bit binary decoder. The binary decoder should take in a 1×4 matrix representing the 4-bit binary numbers as doubles, and output a 1×16 matrix representing the decoder output using one-hot encoding. The output with the highest value should be the correct answer. E.g.

{0.0, 0.0, 1.0, 1.0} should give an output of {0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0 … 0.0}
Note that the brackets here are not indicative of how you should code. Please do take the Matrix's dimension into consideration when coding this.

© Electronics and Computer Science, University of Southampton

Here's a sample screenshot of a test program written for a 3-bit version.

```
**********************************************
Let's test it out!
1
001
******************** OUTPUT ********************
1---0.961375
2---0.000666094
3---0.0445277
4---0.00084229
5---0.0442362
6---2.67252e-07
7---0.000504248
**********************************************
```

For example, when input is 001 (1), the output is [0.961, 0.00, 0.04, 0.00, 0.04, 0.00, 0.00]

```
**********************************************
Let's test it out!
5
101
******************** OUTPUT ********************
1---0.0165159
2---4.53075e-07
3---6.83295e-05
4---0.0162677
5---0.922582
6---6.75226e-05
7---0.0549828
**********************************************
```

When input is 101 (5), then output is [0.01, 0.00, 0.00, 0.01, 0.92, 0.00, 0.05]

3.5 Save weights and test

Save your weights in a file using the fstream class. Modify your program such that you are able to read from pre-trained weights too. (Hint: https://www.w3schools.com/cpp/cpp_files.asp if you forgot how to use fstream!)

4 Optional Work

🚀 **Warning: The optional work here is significantly harder than the lab in general.** Please only attempt this after completing part 3.

Your network works, but so far it only solves a toy problem. Turning a 5-bit binary pattern into a number is something a simple decoder circuit could do!

Here's the **actual challenge**: train a network to recognise items of clothing, using the **same Matrix and ANN classes** you have already written.

For this challenge, we will be using the Fashion MNIST dataset from Zalando Research. The full details of the dataset could be found here: https://www.kaggle.com/datasets/zalando-research/fashionmnist

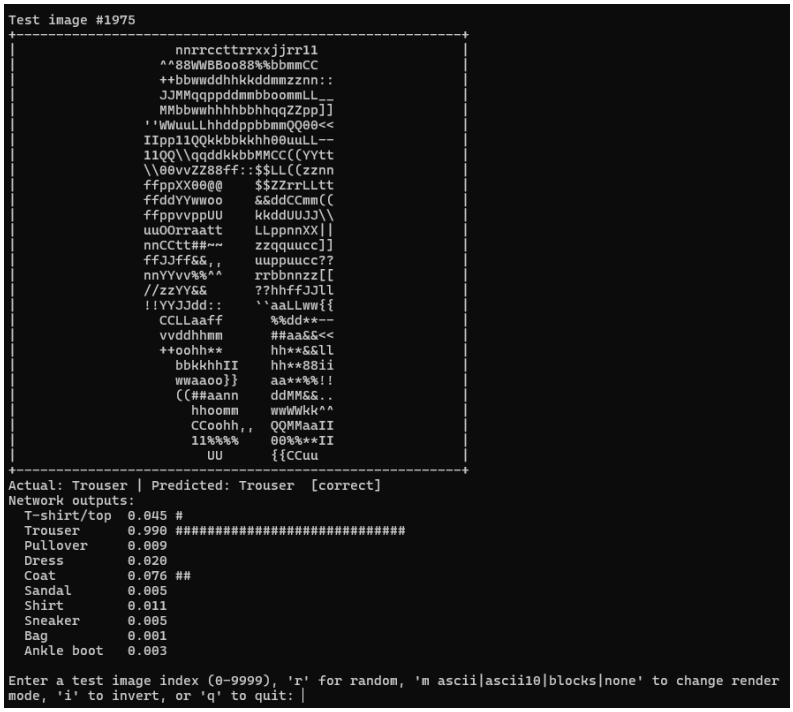
Download the dataset from the "Optional Work" folder. The data contains 2 csv files. One for training, and another one for testing.

© Electronics and Computer Science, University of Southampton

4.1 Load data and train

File format
The first line of each file is a header (label,pixel1,pixel2,…,pixel784). Skip it when reading the data. Every other row is one greyscale image of 28 × 28 pixels, flattened into a single row of 784 values. The pixels are listed row by row, starting at the top-left corner.
Each row has 785 comma-separated integers:

- Column 1 is the label: a whole number from 0 to 9 giving the item's class (see below).
- Columns 2 to 785 are the pixel brightness values, from 0 (black) to 255 (white)



Labels
Each training and test example is assigned to one of the following labels:
0 T-shirt/top
1 Trouser
2 Pullover
3 Dress
4 Coat
5 Sandal
6 Shirt
7 Sneaker
8 Bag
9 Ankle boot

Train your model using the training dataset, test it with the test dataset. Check the accuracy of your model. (It might be helpful to generate some sort of image to help visualize the data, but it is not strictly necessary!)

4.2 Analysis and discussion

Experiment with the architecture. Try changing the structure of your network: change the number of hidden layers and the number of neurons in each layer and see how this affects the test accuracy. Write your code so that the structure can be changed in one place. Document your findings.

References

https://www.tutorialspoint.com/matrix-multiplication-algorithm
https://python-course.eu/machine-learning/neural-networks-structure-weights-and-matrices.php
https://www.thomasbreydo.com/blog/perceptron-cpp

© Electronics and Computer Science, University of Southampton